

Evolution of AI

By

Anand Kulkarni

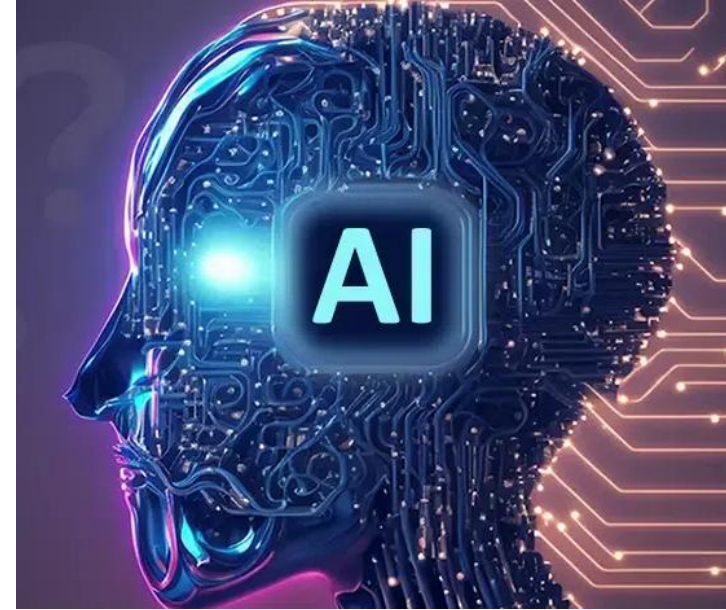
anand.kulkarni1@zensar.com

Table of Content

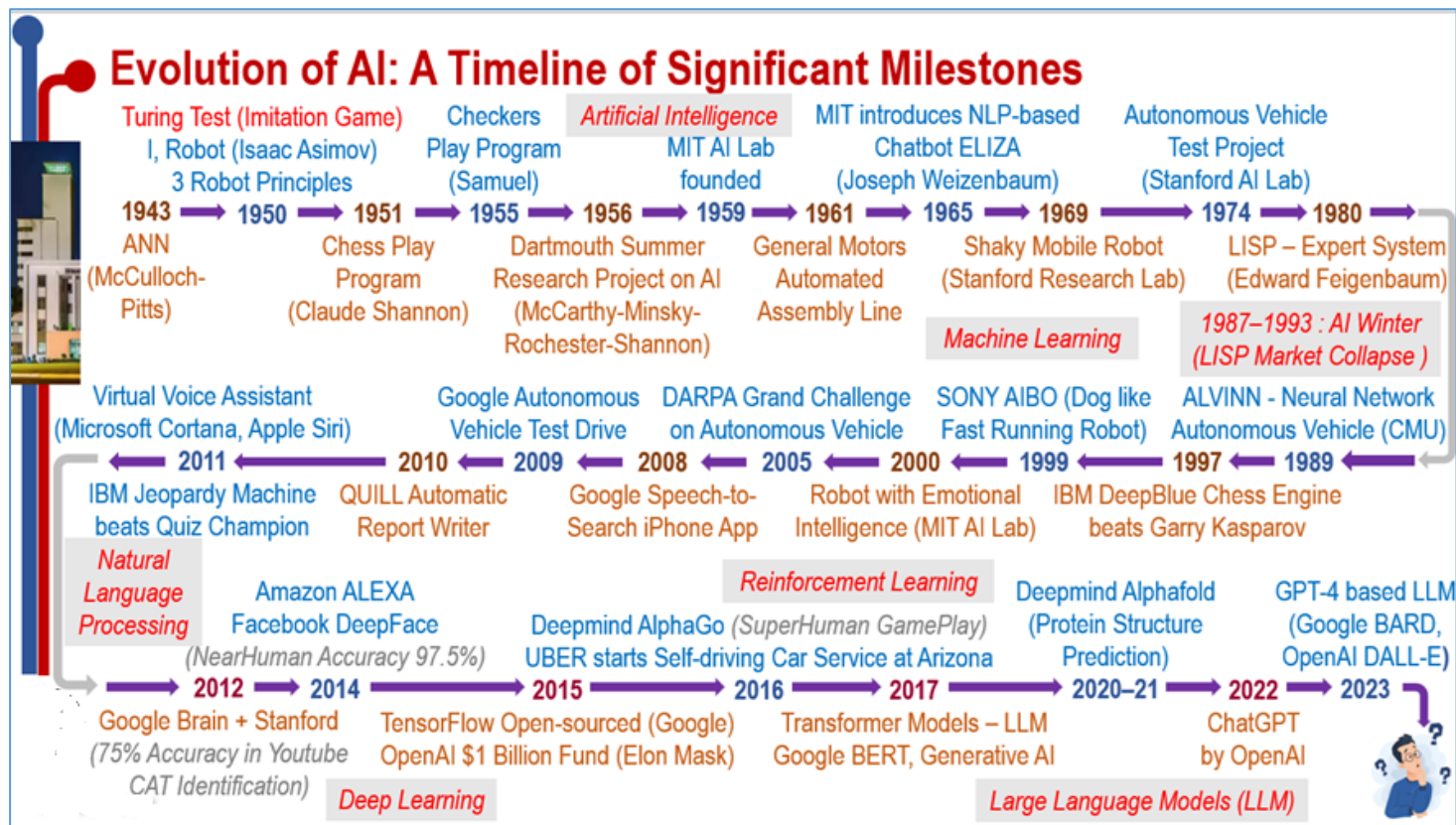
Module	Topic
Module 1:	What is an AI?
Module 2:	AI milestones
Module 3:	AI key components
Module 4:	Machine Learning
Module 5:	Deep Learning
Module 6:	Generative AI
Module 7:	Prompt engineering

What is an AI?

- Artificial Intelligence (AI) refers to the development of computer systems capable of performing tasks that typically require human intelligence.
- These tasks include reasoning, learning, problem-solving, perception, and language understanding.
- AI encompasses a wide range of technologies, including machine learning, deep learning, and natural language processing (NLP).



AI Milestones



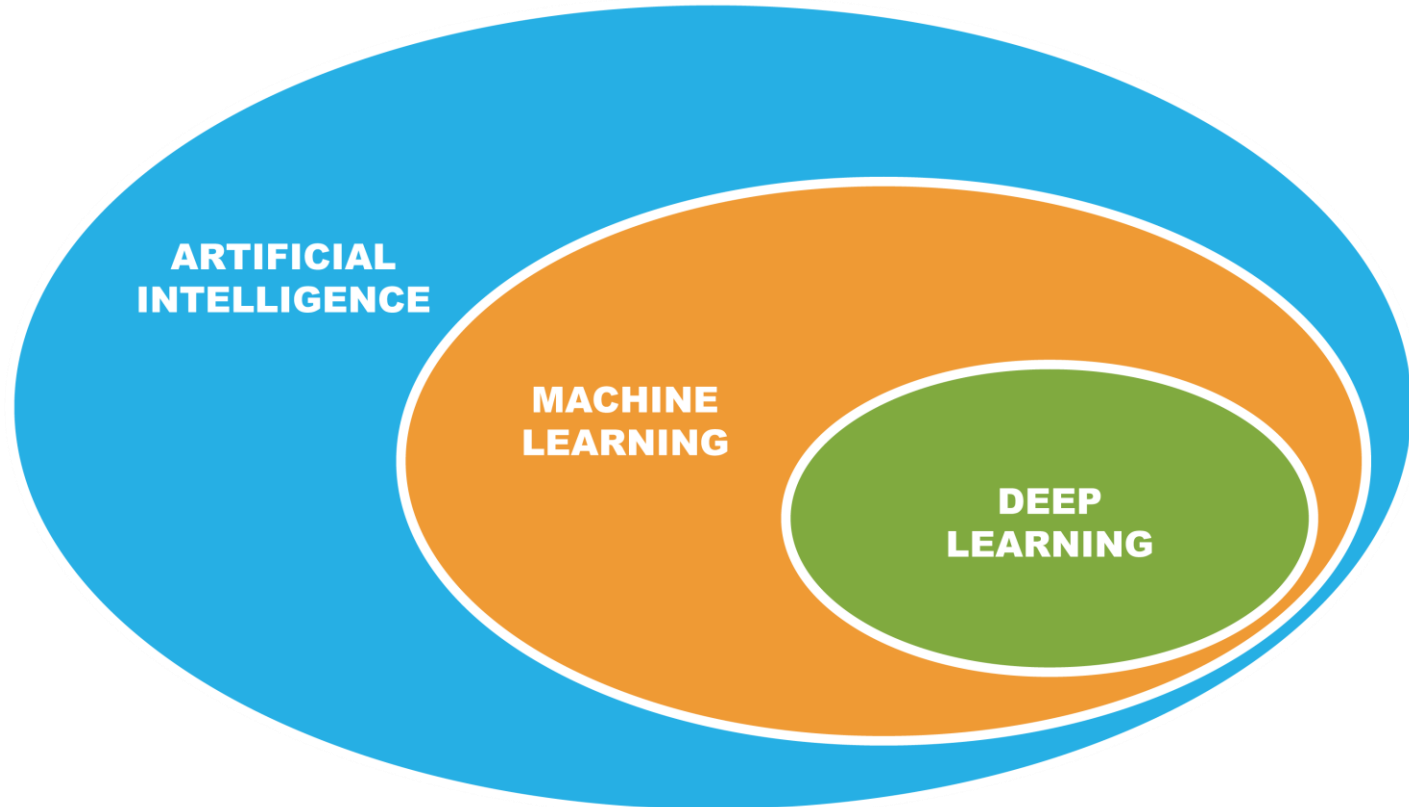
Key AI components

- 1) **Machine Learning:** This involves creating algorithms that allow computers to learn from and make predictions based on data. It includes supervised learning, unsupervised learning, and reinforcement learning. Examples of machine learning include Fraud detection, Spam & Malware filtering, Stock market prediction, Driverless car etc.
- 2) **Deep Learning:** A subset of machine learning that uses neural networks with many layers (hence "deep") to analyze various factors of data. Examples of deep learning include Image segmentation, Algorithmic trading, Speech recognition etc.

Key AI components

- 3) **Natural Language Processing (NLP):** This enables machines to understand and respond to human language. Applications include speech recognition, translation, and sentiment analysis. Examples of NLP are Chatbots, Text autocomplete, Email filtering etc.

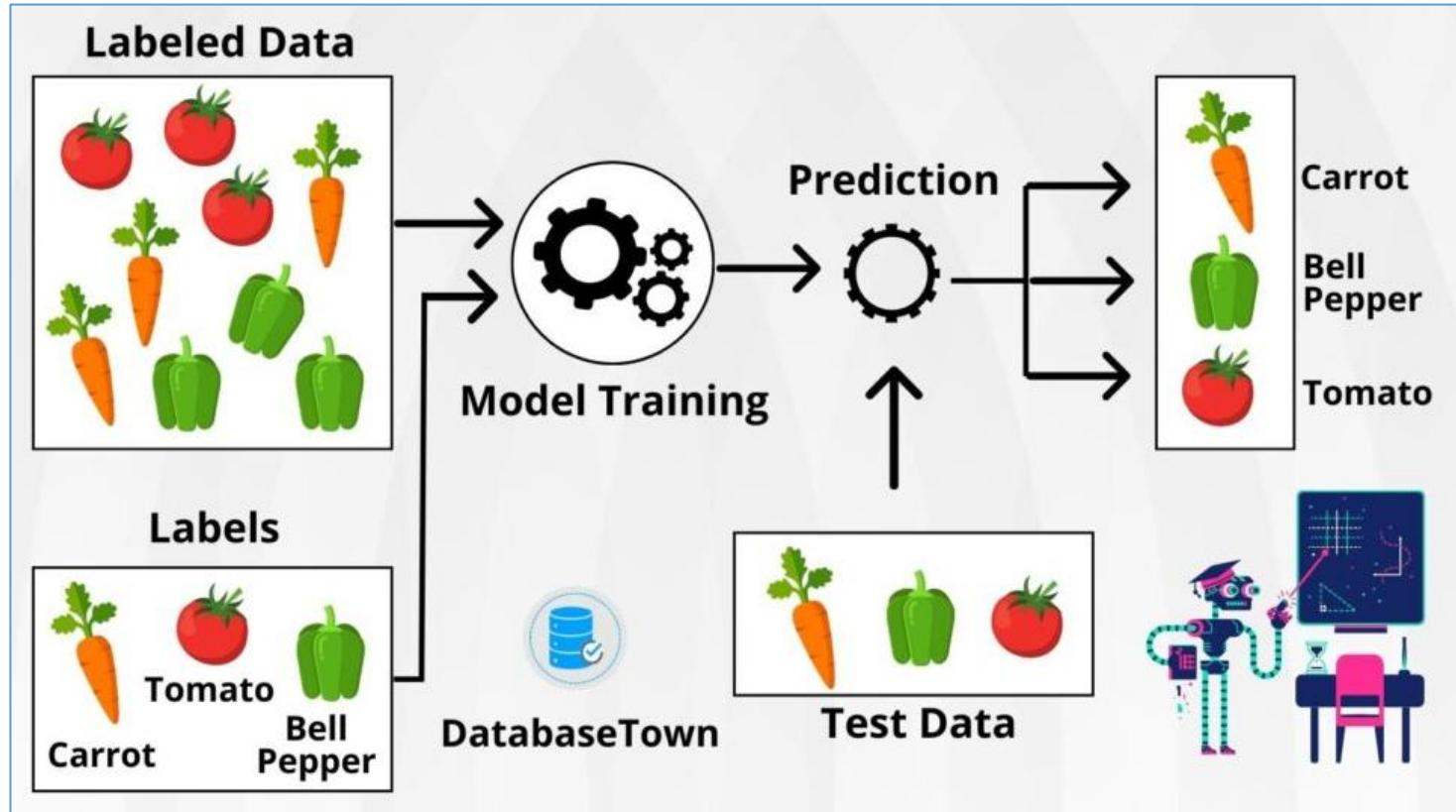
Machine learning and Deep learning



Machine Learning

- Machine Learning is a subset of Artificial Intelligence (AI)
- Systems learn from data and improve automatically
- No need to hard-code rules for every task
- Type of Machine Learning:
 - *Supervised Learning*
 - *Unsupervised Learning*
 - *Reinforcement Learning*

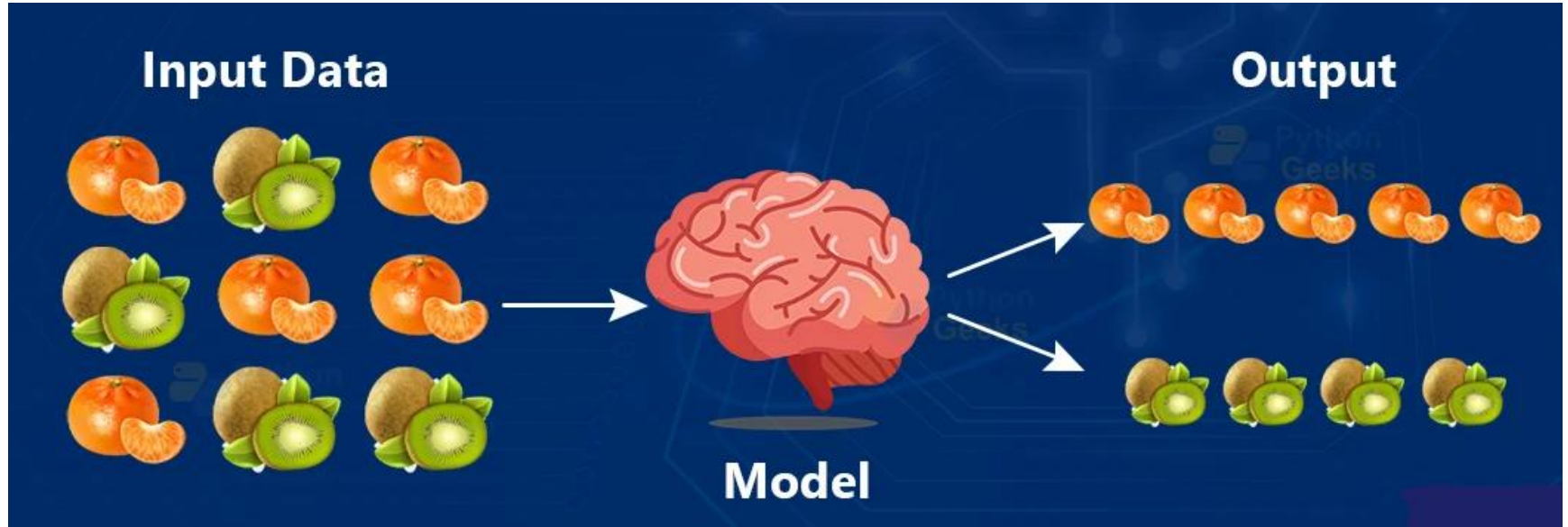
Supervised Learning



Supervised Learning

- Supervised learning uses labeled data (input + correct output)
- Model learns relationship between input and output
- Used for prediction and classification
- Example: House Price Prediction
 - Input: Area, number of rooms, location
 - Model: learns from past labeled data
 - Output: House price
 - *Input Data(Labeled) [Area, Rooms] ---> ML Model ---> Output(Price)*

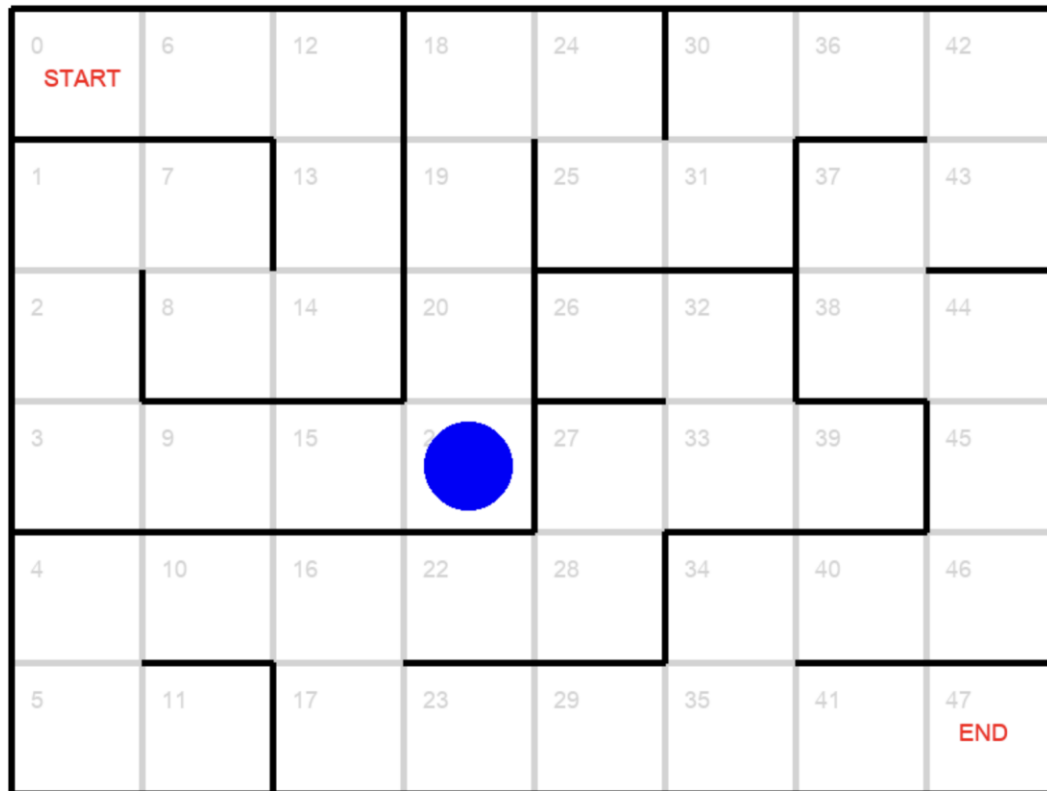
Unsupervised Learning



Unsupervised Learning

- In unsupervised learning, model learns without sample output data i.e. unlabeled data.
- Models are trained on unlabeled data, meaning there are no predefined outcomes or target variables.
- The model's goal is to discover hidden patterns, structures, or relationships within the dataset without explicit guidance.
- Example: Customer Segmentation
 - Input: Customer purchase behavior - No labels provided
 - Output - Customers grouped based on similarity which can be used in marketing
 - *Unlabeled Data[Customer Data] ---> ML Algorithm ---> Groups(Clusters)*

Reinforcement Learning



Reinforcement Learning

- In Reinforcement Learning (RL), an agent learns to make sequential decisions by interacting with an environment to maximize cumulative rewards over time. Unlike supervised learning, RL does not rely on labeled data; instead, it learns through trial-and-error guided by a reward signal.
- Uses rewards and penalties. Hence it is also called as Semi-supervised learning.
- Improves decisions over time
- *Example: Game Playing AI - AI plays a game repeatedly - Reward for winning, penalty for losing - Learns best strategy*

Reinforcement Learning

Applications of Reinforcement Learning



Robotics and Autonomous Systems



Game Playing and Simulation



Healthcare & Personalized Medicine



Finance and Algorithmic Trading



NLP and Dialogue Systems

Machine learning types comparison

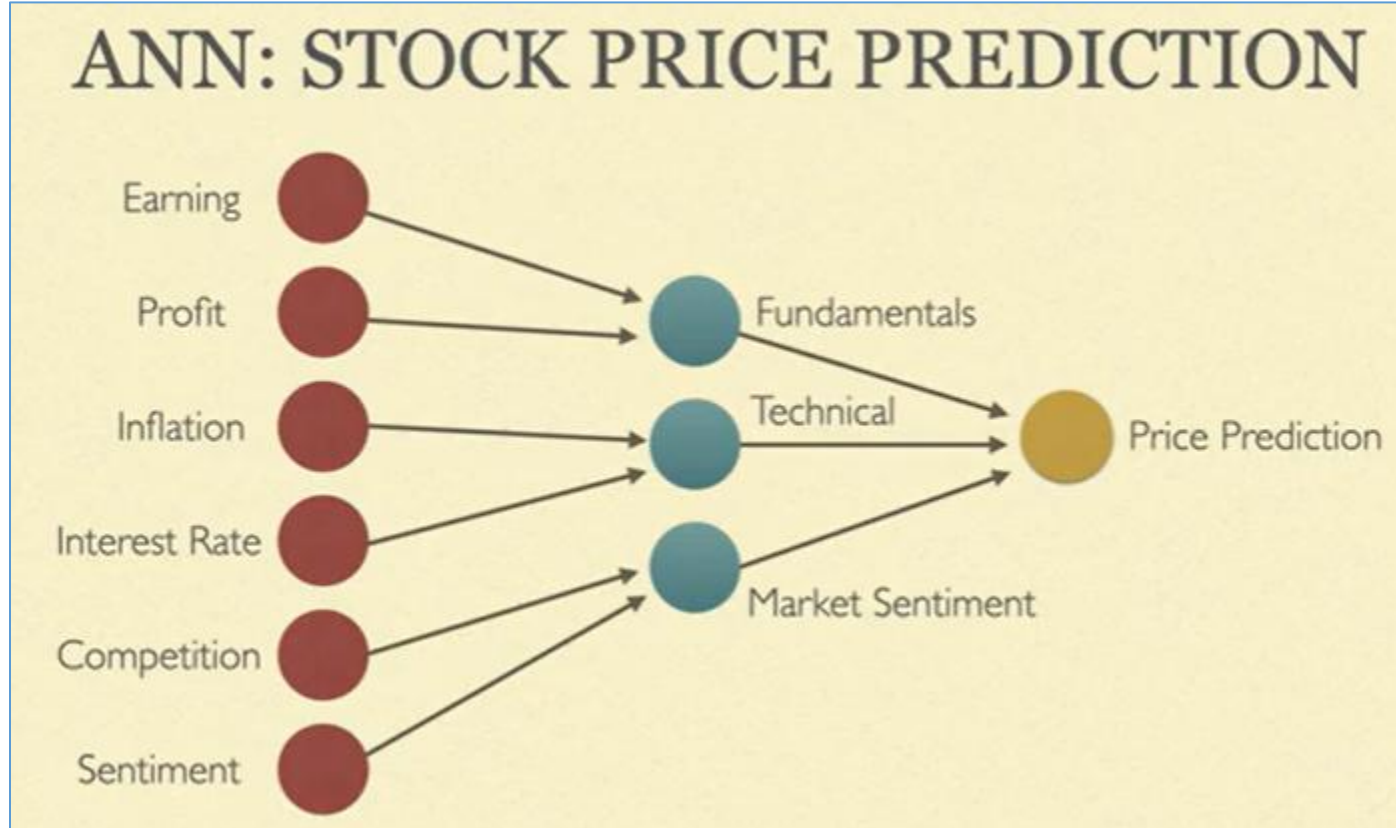
Criteria	Supervised Learning	Reinforcement Learning	Unsupervised Learning
Mapping	Present	Present	Not present
Feedback	Instantaneous feedback once model is created	Constant feedback from environment	No feedback environment
Supervisor	Presence of supervisor and labeled data	No supervisor and labeled dataset is not available	No supervisor
Decisions	Independent and based on training input	Dependent and made sequentially	Independent
Examples	Image classifications	Chess, Robotics	Clustering (i.e. customer segmentation)

Neural Network in human brain



- The human brain has roughly 86 billion neurons interconnected by trillions of synapses (a junction that separates two neurons).
- It forms a web of communication where each neuron processes information through tiny electrical signals called action potentials.
- These signals pass across synapses, chemical bridges between neurons, shaping thoughts, memories, emotions, and actions.

Neural Networks



Neural Networks

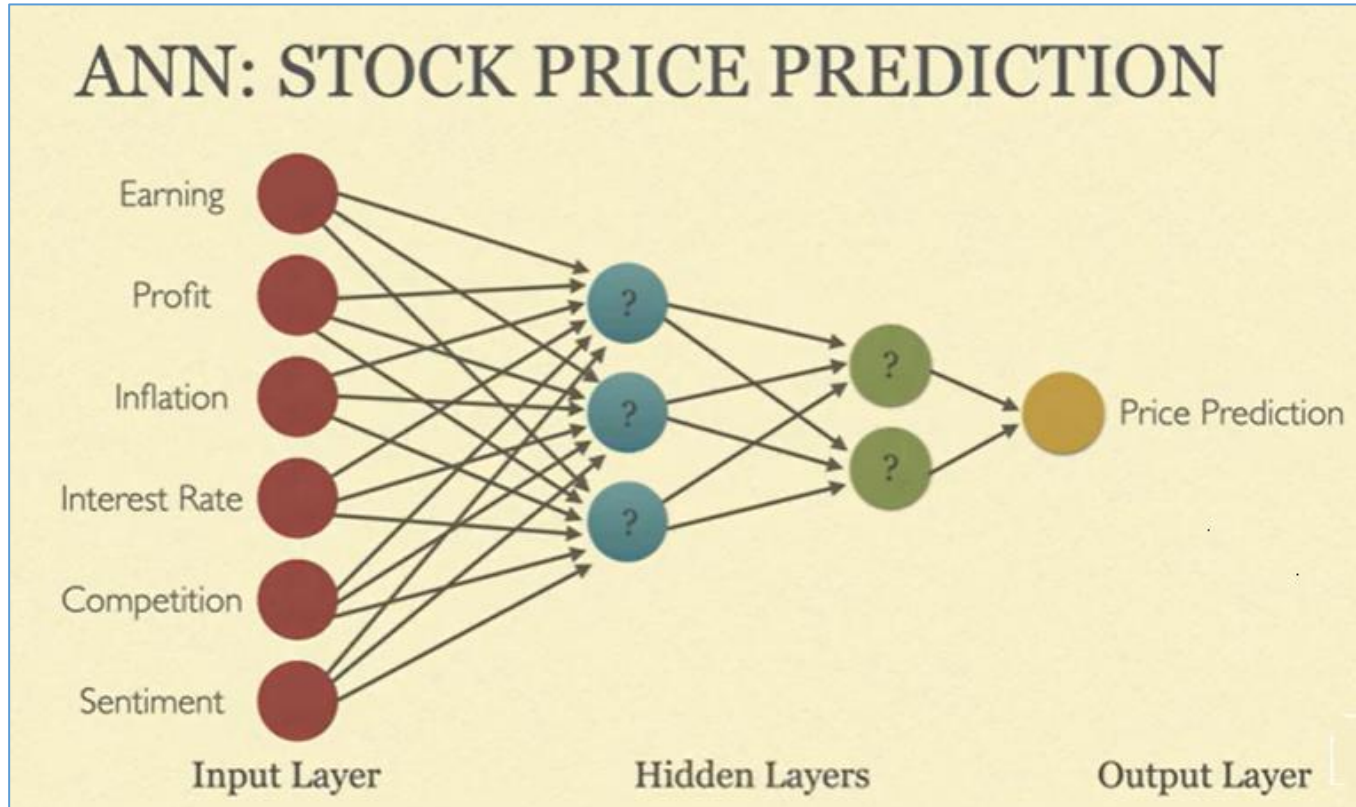
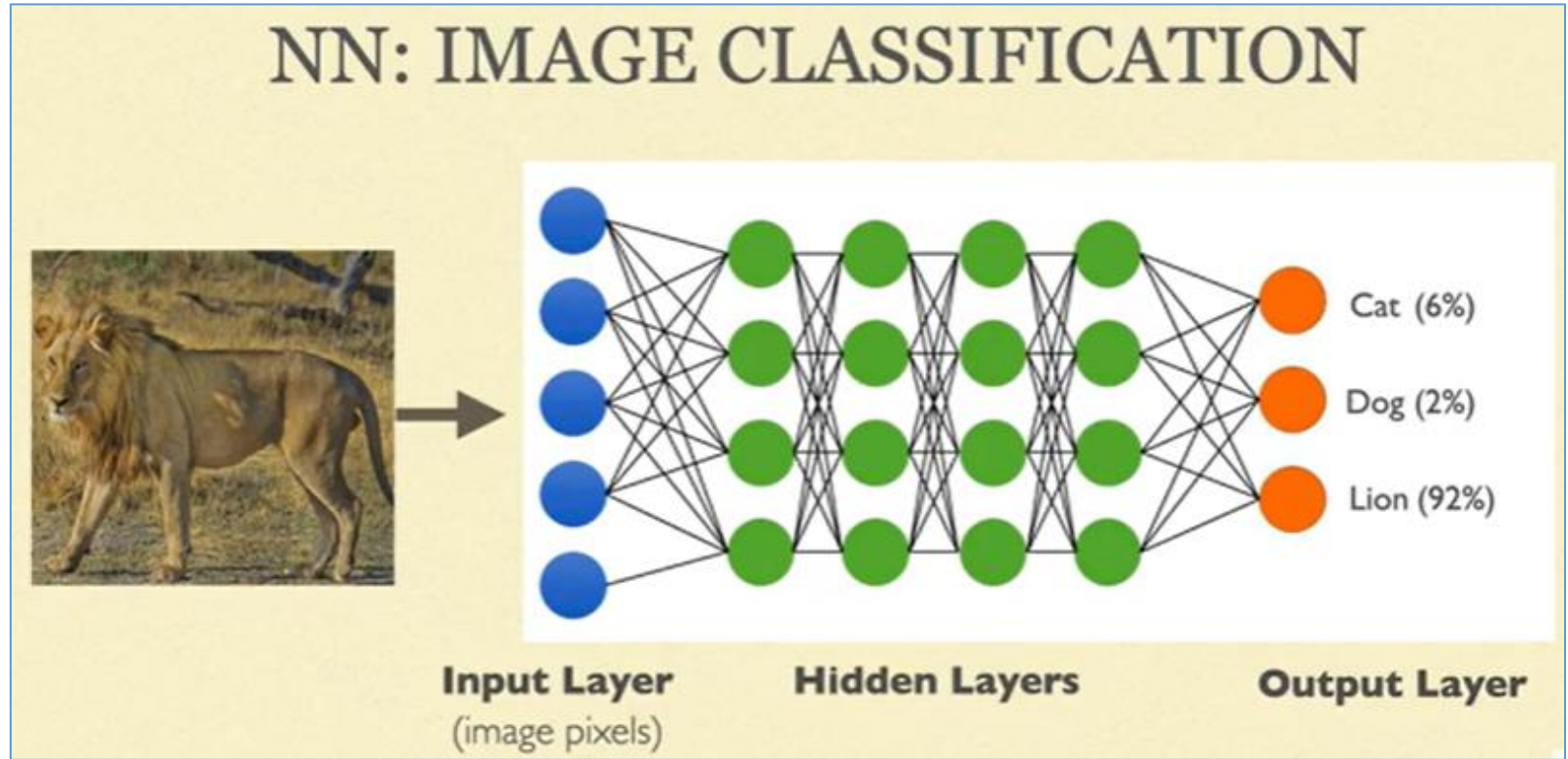


Image classification using Neural Networks



Neural Networks

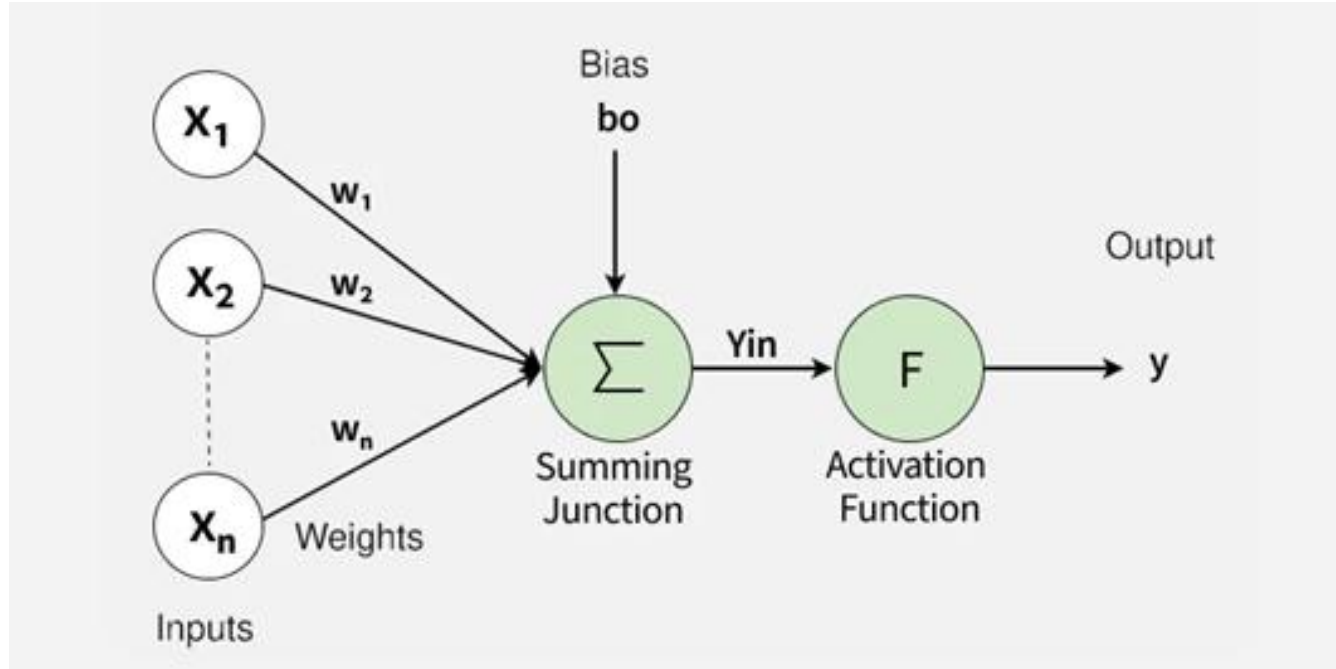
- A neural network is a machine learning model inspired by the human brain.
- It is designed to recognize patterns, make predictions, and solve complex problems.
- In neural networks, below are the core trainable parameters that determine how the model processes inputs and produces outputs:

➤ *Weights*

➤ *Biases*

➤ *Activation function*

How Neural Network works?



Neural Networks

- **Weights:** Weights are numerical values assigned to the connections between neurons. They control the strength and importance of each input feature.
- **Biases:** Biases are additional parameters added to the weighted sum before applying the activation function.
- **Activation function:** Activation function is the component responsible for infusing non-linearity in the neural network. Without it, even a deep network would behave like a simple linear model, limiting its ability to learn complex patterns.
- Neural networks consists of layers of interconnected nodes (neurons), where each neuron processes inputs, applies weights and biases, passes the result through an activation function, and sends the output to the next layer.

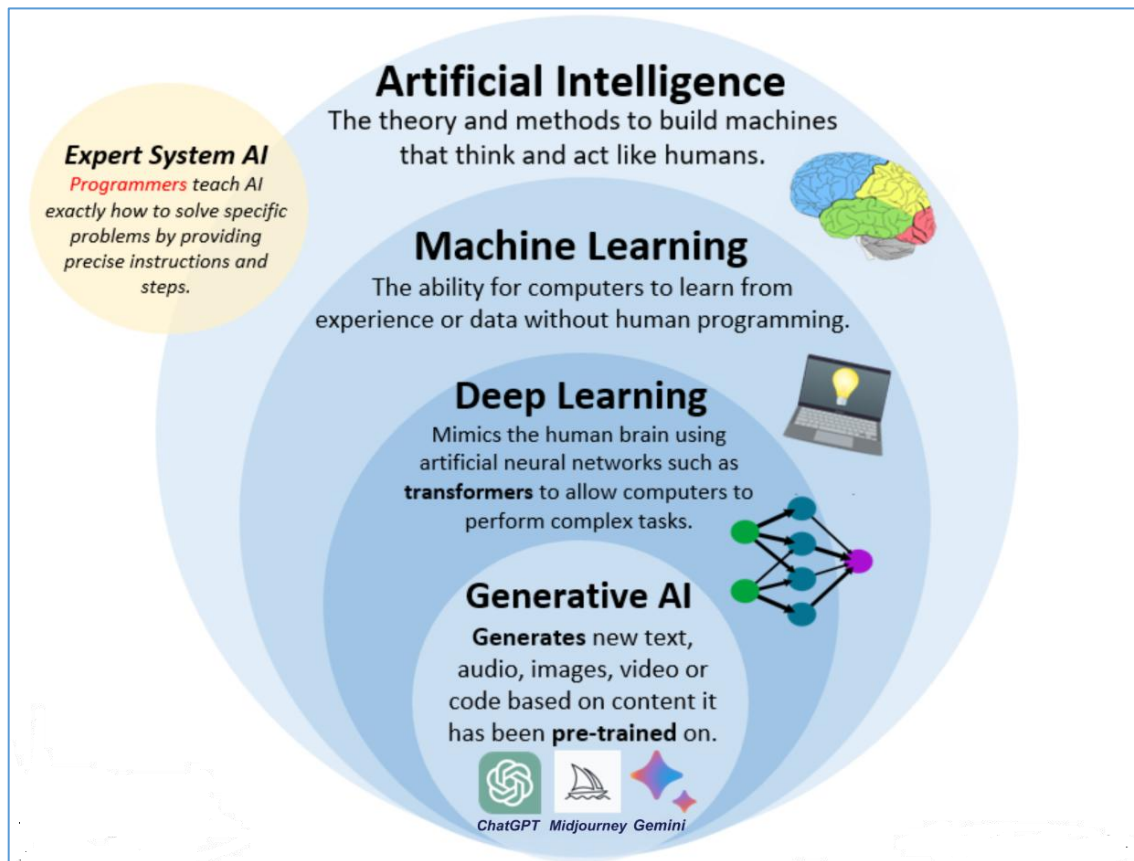
Deep learning

- Deep learning is a subset of machine learning that uses multi-layered artificial neural networks to learn complex patterns from large datasets.
- Deep learning is inspired by the human brain, these networks consist of an input layer, multiple hidden layers, and an output layer, where each neuron applies nonlinear transformations to extract increasingly abstract features from the data.

Deep learning applications



Generative AI



Generative AI applications

Text	Video	Image	Code	Music	Speech	3D
Marketing content	Video generation	Art & design	Code generation	Music composition	Text-to-speech	3D object modeling
Emails	Video editing	Marketing illustrations	Code completion	Sound effect design	Virtual assistance	Architectural visualization
Creative writing	Video game development	Photo editing	Refactoring & optimization	Arrangement & orchestration	Voice cloning	Animations for characters
Translation	Video summarization	Product design & prototyping	Bug detection & fixing	Remixing & mashups	Voice synthesis	Industrial design
Legal & technical texts	VR & AR	Fashion & apparel	Testing	Virtual instruments	Audiobook production	Gaming environments
News articles & summaries		Data visualization	Code formatting		Personalized voice interfaces	

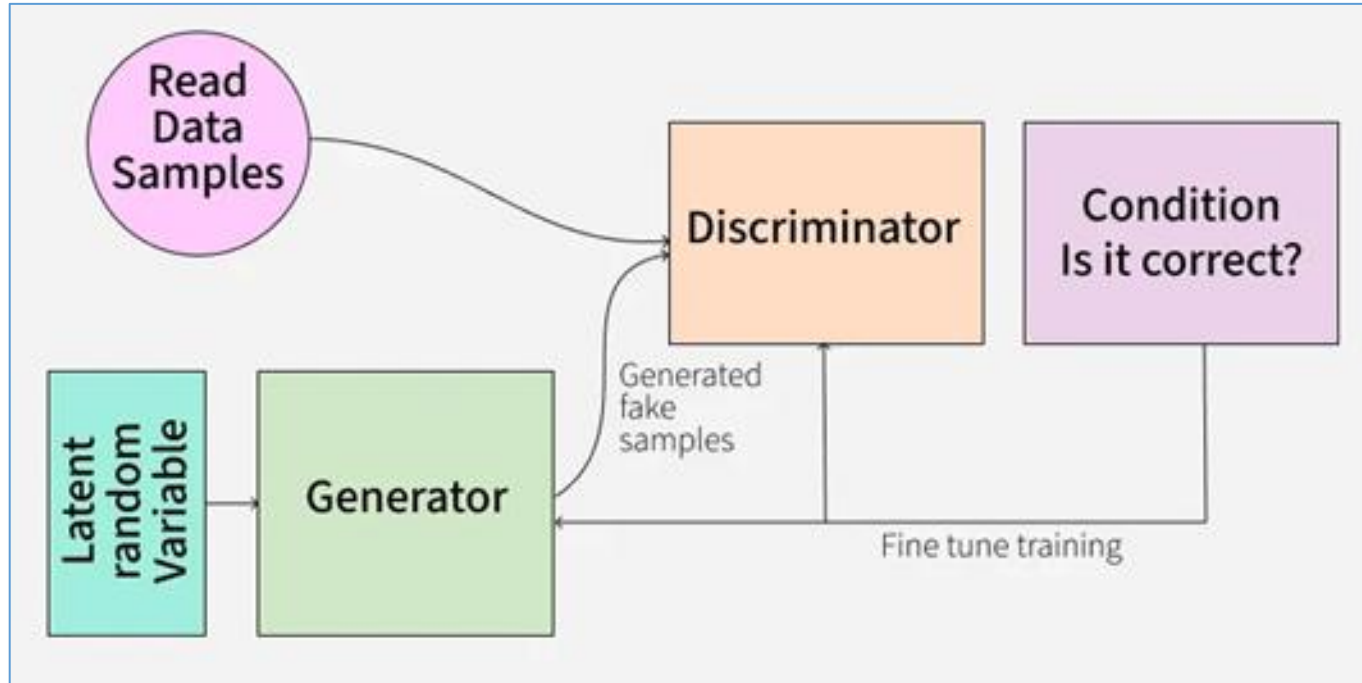
Generative AI (Gen AI)

- Generative AI is a branch of artificial intelligence that focuses on creating new content such as text, images, code, music and video etc.
- Gen AI mainly uses models that are based upon transformers or GANs architecture.
- These models generate original outputs that are often indistinguishable from human-created content.
- These models use techniques like deep learning and neural networks to generate output.
- Gen AI is used in tools like ChatGPT, Claude, Gemini and DALL·E and is widely applied in automation, chatbots and personalization.

Gen AI Models (GAN vs Transformer)

- Two architectures are widely used in the Generative models, Generative Adversarial Networks(GANs) and Transformer models.
- Both excel at generating data, but they operate on different principles.

GAN (Generative Adversarial Networks)



GAN (Generative Adversarial Networks)

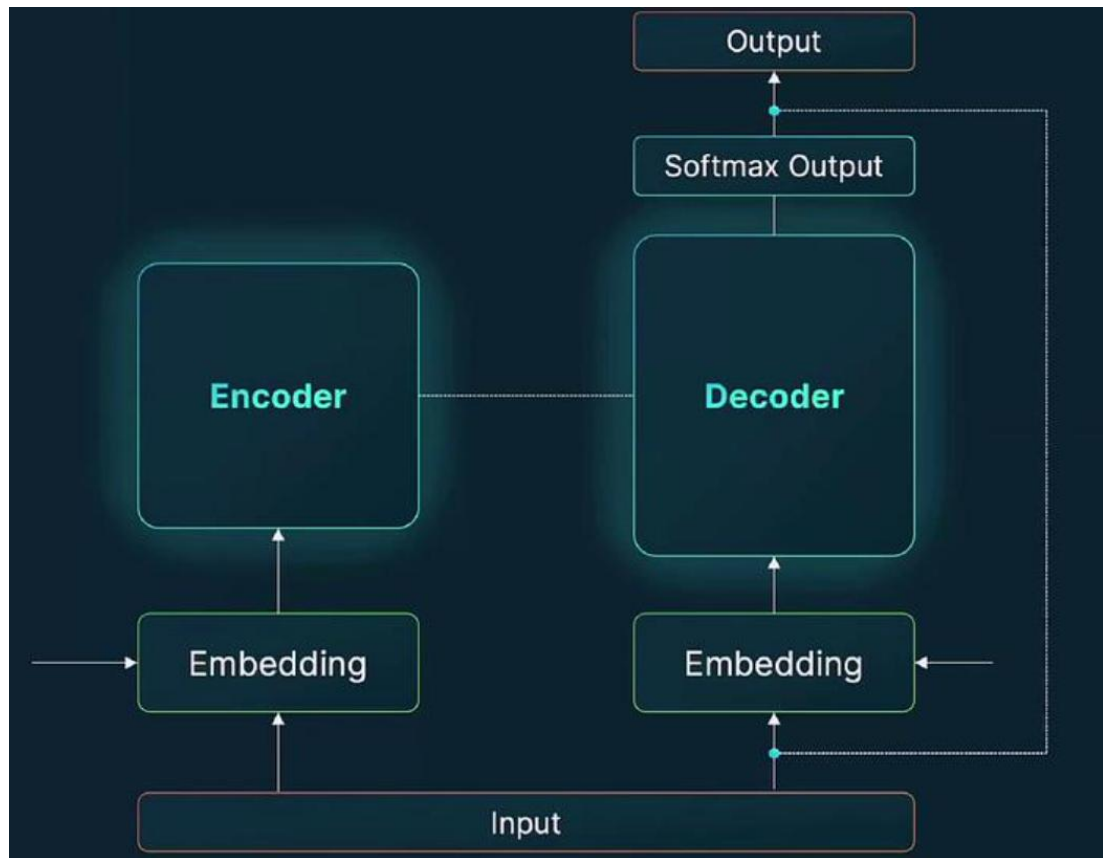
- GANs are a framework consisting of two competing neural networks: a generator that creates fake data and a discriminator that tries to differentiate between real and fake data.
- The generator learns to produce increasingly realistic data by trying to fool the discriminator, while the discriminator becomes better at detecting fake data.
- This adversarial training process continues until the generator produces data so realistic that the discriminator can barely tell the difference from real data.

GAN (Generative Adversarial Networks)

GANs consist of two neural networks trained in opposition to one another:

- **Generator:** Produces synthetic data that mimics the distribution of real training data.
- **Discriminator:** Attempts to distinguish between real and generated (fake) samples.

Transformer Architecture



Transformer Architecture

Transformer is a neural network architecture that has fundamentally changed the approach to Artificial Intelligence.

It was introduced in 2017.

Every text-generative Transformer consists of these three key components:

Embedding: Text input is divided into smaller units called tokens, which can be words or sub-words. These tokens are converted into numerical vectors called embeddings, which capture the semantic meaning of words.

Transformer Architecture

Transformer Block: It is the fundamental building block of the model that processes and transforms the input data. Each block includes Attention Mechanism & MLP (Multilayer Perceptron) Layer. Attention Mechanism the core component of the Transformer block. It allows tokens to communicate with other tokens, capturing contextual information and relationships between words. MLP is a feed-forward network that operates on each token independently. While the goal of the attention layer is to route information between tokens, the goal of the MLP is to refine each token's representation.

Transformer Architecture

Output Probabilities: The final linear and softmax layers transform the processed embeddings into probabilities, enabling the model to make predictions about the next token in a sequence.

The softmax function is a crucial component of the transformer architecture, particularly in the context of the attention mechanism. It converts the scores computed by the self-attention mechanism into probability distributions. This function is essential for two reasons: it provides meaningful scores and allows the model to make informed decisions during generation tasks like machine translation or text summarization. By converting raw scores into probabilities, softmax ensures that the model can understand the relative importance of words in a sequence and select the next word with the highest probability.³⁶

Prompt engineering

Prompt engineering is the process of writing, refining, and optimizing inputs to guide generative AI systems in producing specific, high-quality outputs.

Single: Always focus your prompt on a single, well-defined task or question. This clarity is crucial for accurate and useful responses from Copilot.

Specific: Ensure that your instructions are explicit and detailed. Specificity leads to more applicable and precise code suggestions.

Short: While being specific, keep prompts concise and to the point. This balance ensures clarity without overloading Copilot or complicating the interaction.

Surround: Utilize descriptive filenames and keep related files open. This provides Copilot with rich context, leading to more tailored code suggestions.

How Model Learns from your Prompts?

AI models trained on vast amounts of data. To enhance its understanding of specific code contexts, engineers often provide it with examples. This practice, commonly found in machine learning, led to different training approaches such as:

- Zero-shot learning
- One-shot learning
- Few-shot learning

How Model Learns from your Prompts?

- Zero-shot learning

Here, model generates output without any specific examples, relying solely on its foundational training. For example,

I need a function to convert Celsius to Fahrenheit

def celsius_to_fahrenheit(celsius):

*fahrenheit = (celsius * 9/5) + 32*

return fahrenheit

How Model Learns from your Prompts?

- One-shot learning

With this approach, a single example is given, aiding the model in generating a more context-aware response.

Example: Function to convert Fahrenheit to Celsius

```
def fahrenheit_to_celsius(fahrenheit):
```

```
    celsius = (fahrenheit-32) * 5/9
```

```
    return celsius
```

Now I need a function to convert Celsius to Fahrenheit

How Model Learns from your Prompts?

```
def celsius_to_fahrenheit(celsius):
```

```
    fahrenheit = celsius * 9/5 + 32
```

```
    return fahrenheit
```

How Model Learns from your Prompts?

- Few-shot learning

In this method, Model is presented with several examples, which strike a balance between zero-shot unpredictability and the precision of fine-tuning.

Example 1: Greeting message for morning

Input: 9 AM

Output: "Good morning!"

Example 2: Greeting message for afternoon

Input: 2 PM

Output: "Good afternoon!"

How Model Learns from your Prompts?

- Few-shot learning

Example 3: Greeting message for evening

Input : 7 PM

Output : "Good evening!"

Now, generate a python code that takes the current time as input using the datetime module and returns the appropriate greeting message

Thank you!!